# BCL1213 Operating Systems

## Mock Examination: Answers and Marking Scheme

**Chapters 1 to 4 | 100 marks | 11 November 2026**
**Prepared for:** Chan Jing Yi
**Student ID:** SUOL2500321

This key accompanies the independently prepared practice paper. Equivalent technically correct wording is acceptable. Award method marks where a correct method is followed with a later arithmetic error; do not award the same conceptual point twice. Suggested allocations are for self assessment, not an official institutional marking rubric.

# Section A: Answer Key [20 marks]

Award 1 mark for each correct answer and 0 otherwise.

| Question | Answer | Explanation |
|---|---|---|
| 1 | D | The OS coordinates resources and provides controlled services to programs. |
| 2 | D | A driver implements device specific control behind the operating system interface. |
| 3 | C | A monolithic kernel integrates many OS services in the privileged kernel environment. |
| 4 | C | Time sharing aims for responsive interaction. Multiprogramming primarily keeps useful CPU work available during I/O waits. |
| 5 | A | A hard real time requirement makes the deadline part of correctness; high average speed alone is insufficient. |
| 6 | D | The event makes the process eligible for CPU service. It must still be dispatched. |
| 7 | D | Saved execution context is part of process bookkeeping used when resuming execution. |
| 8 | C | Short quanta may improve responsiveness but increase switching overhead. |
| 9 | A | Turnaround is 11 − 2 = 9 ms. Waiting is 9 − 6 = 3 ms. |
| 10 | C | Threads share the process address space while retaining their own execution context and stacks. |
| 11 | A | Registers and cache are faster and smaller than main memory; secondary storage is slower. |
| 12 | B | Internal fragmentation is waste within an allocation; external fragmentation concerns scattered free holes. |
| 13 | B | Pages are logical units and frames are corresponding physical units of the same size. |
| 14 | B | For a length of 500, valid offsets range from 0 through 499. |
| 15 | B | Thrashing occurs when heavy page movement dominates useful computation. |
| 16 | B | An unprotected shared update can produce different outcomes for different interleavings. |
| 17 | D | A mutex has an owner, unlike a semaphore used for permit counting or signaling. |
| 18 | A | Both permits have been consumed. A further acquire must wait. |
| 19 | A | The waiter reacquires the mutex before returning and rechecks the predicate because the state may have changed. |
| 20 | C | A global acquisition order prevents a cycle of lock dependencies. |

# Section B: Model Answers

## Question 21: OS Fundamentals and Architecture [20 marks]

### (a) Definition and responsibilities [5 marks]

An operating system is system software that manages computer resources and provides services and controlled interfaces for applications. CPU management selects a ready process for execution and allocates processor time. Memory management tracks allocations, translates addresses where supported, and protects process memory from inappropriate access.

**Marks:** A sound definition, 1 mark. Identification and explanation of each of two responsibilities, 2 marks each. File management, device management or protection may replace either example if correctly explained.

### (b) Printing request [4 marks]

The application requests a printing service through the operating system interface. The OS coordinates the request and any print queue. The printer driver translates the generic request into commands or data suitable for that printer. The printer receives the device specific instructions and produces output, with status or completion information returned through the system.

**Marks:** Application request, 1 mark; OS coordination, 1 mark; driver translation, 1 mark; hardware execution or completion/status, 1 mark. A sensible explanation may mention spooling but does not need to describe its internal implementation.

### (c) Monolithic and microkernel comparison [6 marks]

A monolithic design places many services, such as core memory, file and device operations, in kernel space. A microkernel retains a smaller privileged core and runs more services outside the kernel, commonly as user space processes that communicate through messages. Keeping services in one privileged environment can reduce communication overhead. Moving services outside the kernel can improve separation and limit the impact of some service failures, although communication and context changes may add overhead. Neither design automatically guarantees superior performance or complete fault isolation in every implementation.

**Marks:** Monolithic placement, 2 marks; microkernel placement, 2 marks; one explained trade off linking design to overhead or isolation, 2 marks.

### (d) Hospital processing models [5 marks]

Batch processing suits overnight billing because jobs can be collected and completed without continuous interactive responses. A hard real time processing model suits the controller because it must react before a mandatory deadline. Fast average performance does not establish a worst case response bound; occasional long delays could still violate the deadline even if most requests finish quickly.

**Marks:** Billing model and reason, 2 marks; controller model and deadline justification, 2 marks; average versus worst case distinction, 1 mark. These are task requirements; a single hospital may use several systems rather than one OS for every task.

# Question 22: Process Management and Scheduling [20 marks]

### (a) FCFS [8 marks]

FCFS executes P1, then P2, then P3 in arrival order.

```
0          6       9   10
|    P1    |  P2   | P3 |
```

| Process | Completion, CT (ms) | Turnaround, CT minus arrival (ms) | Waiting, turnaround minus burst (ms) |
|---|---|---|---|
| P1 | 6 | 6 minus 0 = 6 | 6 minus 6 = 0 |
| P2 | 9 | 9 minus 1 = 8 | 8 minus 3 = 5 |
| P3 | 10 | 10 minus 2 = 8 | 8 minus 1 = 7 |

Average waiting time = (0 + 5 + 7) / 3 = **4 ms**.

**Marks:** Correct Gantt sequence and boundaries, 2 marks; completion times, 1 mark; turnaround values or consistently correct formula and application, 2 marks; waiting values or consistently correct formula and application, 2 marks; average, 1 mark.

### (b) Round Robin, quantum 2 ms [8 marks]

At time 2, P2 is already waiting. P3 arrives at that boundary and joins before P1 is requeued, so the ready queue is P2, P3, P1.

```
0    2    4   5    7   8    10
| P1 | P2 | P3 | P1 | P2 | P1 |
```

| Interval (ms) | CPU | Result at interval end |
|---|---|---|
| 0 to 2 | P1 | 4 ms remaining; queue becomes P2, P3, P1 |
| 2 to 4 | P2 | 1 ms remaining; queue becomes P3, P1, P2 |
| 4 to 5 | P3 | Completes; queue becomes P1, P2 |
| 5 to 7 | P1 | 2 ms remaining; queue becomes P2, P1 |
| 7 to 8 | P2 | Completes; P1 remains |
| 8 to 10 | P1 | Completes |

| Process | CT (ms) | Turnaround (ms) | Waiting (ms) |
|---|---|---|---|
| P1 | 10 | 10 minus 0 = 10 | 10 minus 6 = 4 |
| P2 | 8 | 8 minus 1 = 7 | 7 minus 3 = 4 |
| P3 | 5 | 5 minus 2 = 3 | 3 minus 1 = 2 |

Average waiting time = (4 + 4 + 2) / 3 = 10 / 3 = **3.33 ms**, rounded to two decimal places.

**Marks:** Gantt sequence, boundaries and boundary arrival ordering, 3 marks; completion times, 1 mark; waiting calculations, 3 marks; average, 1 mark. Turnaround values support the working but receive no extra marks beyond this allocation.

### (c) States and PCB [4 marks]

A Ready process can execute as soon as it receives CPU time. A Waiting process cannot yet execute because an event, such as I/O completion, has not occurred. During a context switch, the OS saves execution state such as the program counter and registers in process context bookkeeping, represented by the PCB in the course model. It restores the selected process context so that execution can resume correctly.

**Marks:** Ready distinction, 1 mark; Waiting distinction, 1 mark; saving context, 1 mark; restoring/resuming context, 1 mark.

# Question 23: Memory Management [20 marks]

### (a) Paging [6 marks]

1. Page number = integer quotient of 2500 / 1024 = **2**.
2. Offset = 2500 modulo 1024 = 2500 minus 2048 = **452 bytes**.
3. Validate the offset: **0 ≤ 452 < 1024**, so it fits within one page.
4. Page 2 maps to **frame 7**.
5. Physical address = frame number × page size + offset = 7 × 1024 + 452 = 7168 + 452 = **7620 bytes**.

**Marks:** Page number, 1 mark; offset, 1 mark; validity check, 1 mark; correct frame lookup, 1 mark; formula, 1 mark; final physical address, 1 mark.

### (b) Segmentation [6 marks]

A reference is valid only when 0 ≤ offset < limit, since the limit is the segment length. Translate a valid reference using base + offset.

| Reference | Validity check | Outcome |
|---|---|---|
| Segment 0, offset 200 | 0 ≤ 200 < 400 | Valid; physical address = 1000 + 200 = **1200** |
| Segment 1, offset 600 | 600 equals the length 600 | Invalid; maximum valid offset is 599; no physical address is produced |
| Segment 2, offset 299 | 0 ≤ 299 < 300 | Valid; physical address = 4000 + 299 = **4299** |

**Marks:** 2 marks per reference, consisting of 1 mark for the correct validity decision and 1 mark for the valid calculation or invalid boundary explanation. Do not award a translation mark for adding the base to an invalid offset.

### (c) Independent allocation choices [6 marks]

| Method | Choice and reason | Remaining space |
|---|---|---:|
| First Fit | The first sufficient hole in address order is 500 KiB | 500 minus 212 = **288 KiB** |
| Best Fit | The smallest sufficient hole is 300 KiB | 300 minus 212 = **88 KiB** |
| Worst Fit | The largest available hole is 600 KiB | 600 minus 212 = **388 KiB** |

**Marks:** Correct hole, 1 mark per method; correct remainder, 1 mark per method. Evaluate every method from the original list. Do not carry one method's allocation into the next method's calculation.

### (d) Fragmentation [2 marks]

Internal fragmentation is unused space inside an allocated block. External fragmentation is free space scattered across separate holes, so a sufficiently large contiguous request may fail even when the total free space is sufficient.

**Marks:** 1 mark for each accurate distinction.

# Question 24: Synchronization and Deadlocks [20 marks]

### (a) Lost update and mutex protection [6 marks]

Both threads read 10. A computes 11 and B computes 9. A writes 11 first, then B overwrites it with 9. The final counter is therefore **9**, whereas serialized execution of both updates gives **10**. This is a race condition producing a lost update. Each thread must acquire the same mutex before reading the shared counter, perform the entire update while holding the mutex, and release it after writing. Locking only the final write would leave the read and computation vulnerable to interference.

**Marks:** Final value with trace or explanation, 2 marks; expected serialized value, 1 mark; identification of race condition or lost update, 1 mark; same mutex covering the entire update, 2 marks.

### (b) Semaphore and ownership [4 marks]

Each successful acquire consumes one permit. Three threads can therefore obtain one connection each, reducing the available permit count from 3 to 0. A fourth acquire waits until a connection is released and a permit is returned. The user of a connection must return its permit once, after finishing with that connection. A semaphore has no mutex style ownership rule and can support signaling by another thread under the program protocol. A mutex, by contrast, must be released by its acquiring owner.

**Marks:** Permit consumption and correspondence to three resources, 1 mark; fourth requester waits, 1 mark; release restores a permit, 1 mark; ownership distinction, 1 mark.

### (c) Condition variable [4 marks]

The consumer acquires the mutex that protects the queue and tests whether the queue is empty. While it is empty, the consumer waits on the condition variable. The wait atomically releases that mutex while blocking and reacquires it before returning. The consumer tests the predicate again in a loop because a notification does not guarantee that an item remains available, and a wakeup may occur without the desired condition being true.

```
lock(queue_mutex)
while queue_is_empty:
    wait(not_empty, queue_mutex)
item = remove_item()
unlock(queue_mutex)
```

**Marks:** Protected predicate check with the mutex, 1 mark; atomic release while waiting, 1 mark; reacquisition before continuing, 1 mark; loop with a valid reason, 1 mark.

### (d) Deadlock and global ordering [6 marks]

The four necessary conditions are **mutual exclusion**, **hold and wait**, **no preemption** and **circular wait**. In this example, X waits for the lock held by Y while Y waits for the lock held by X. Require every thread to acquire A before B. A thread cannot then hold B while waiting to acquire A, so this circular dependency cannot form. The global ordering rule breaks circular wait.

**Marks:** Correct condition names, 4 marks total; consistent order with an explanation that it breaks circular wait, 2 marks. The conditions are necessary for resource deadlock; their general presence alone is not proof that every current allocation involving multiple resource instances is already deadlocked.

# Score Record and Revision Map

| Component | Maximum | Your score |
|---|---:|---|
| Section A | 20 | |
| Question 21 | 20 | |
| Question 22 | 20 | |
| Question 23 | 20 | |
| Question 24 | 20 | |
| **Total** | **100** | |

Calculate each chapter score out of 25: Questions 1 to 5 plus Q21 for Chapter 1; Questions 6 to 10 plus Q22 for Chapter 2; Questions 11 to 15 plus Q23 for Chapter 3; Questions 16 to 20 plus Q24 for Chapter 4. Revise missed concepts and attempt them again without reading the model answer first. No institutional grade boundary is implied by this practice score.

## Source basis

The topics were checked against the original course materials: Ruffinus Rusell, *Chapter 01 Introduction to Operating Systems*; *Chapter 02 Process Management*; *Process Management and Scheduling*, Chapter 2 SIM; *Chapter 03 Memory Management*, Parts A and B; *Memory Management*, Chapter 3 SIM; and *Process Synchronization*, Chapter 4 SIM. These course materials are undated. Questions, datasets and mark allocations in this paper are newly prepared practice material.

Offset bounds and units are stated explicitly. Inconsistent source examples are not reproduced as valid calculations.